Revisión Crítica - TP Skills y MCP (Punto 8)

Evidencia de la invocación de la skill y de la herramienta del MCP, y defe

Proyecto

SaaS multi-tenant de gestión para consultorios odontológicos (Argentina)

Fecha

2026-10-19

Alcance de esta revisión

La consigna del punto 8 pide: guardar evidencia de la invocación de la skill y de la herramienta del MCP (solicitud, resultado y qué aportó al change), e identificar al menos una omisión, error o sugerencia inadecuada en los resultados del agente, explicando cómo se detectó, qué fuente o regla permitió corregirla y mostrando el antes y el después.
Esta revisión se aplica en dos frentes: el **change C-02** (el de la consigna del punto 7, preparado para implementar) y la **ejecución de C-01** (el change implementado y archivado), porque en C-01 el agente también produjo resultados que había que revisar.
Informes detallados: `revision-critica-punto-8-c02.md` (C-02) y `revision-critica-punto-8.md` (C-01).

1. Evidencia de la invocación de la skill

Solicitud realizada: `skill({ id: "visual-explore-options" })`, en el turno en que se exploró C-02.
Resultado obtenido: el `SKILL.md` completo, con su procedimiento (contexto -> restricciones -> >=2 alternativas -> esquema ASCII -> plantilla exacta -> pausa obligatoria) y sus criterios de calidad. El resultado operativo es el artefacto `discovery/explore-c02-tenancy-core-models.md`: 4 ejes con 2 opciones cada uno, alternativas descartadas con su razonamiento, esquemas ASCII, recomendación y la tabla de decisiones tomadas (§0).
Qué aportó al change:
- Forzó la comparación explícita de alternativas **antes** de fijar el esquema. El eje 3 (repositorio tenant-scoped vs. `tenant_id` explícito) es la decisión que convierte R1 -la regla dura #1 del proyecto- en estructura en vez de disciplina; sin la comparación, el patrón del slice archivado se habría replicado por inercia.
- Detectó y cerró la colisión entre la migración `002` y la `0001` ya existente, dejando por escrito que `0001` no se toca.
- Puso en pausa la escritura de código justo donde hacía falta: el usuario eligió las 4 opciones **antes** de que existiera un solo archivo de `design.md`.
- Dejó el porqué de cada alternativa descartada, que es lo que permite cuestionar la decisión más adelante. Por ejemplo, la tabla puente de roles quedó descartada **porque contradice la KB §04**, no por preferencia.

2. Evidencia de la invocación de la herramienta del MCP

Servidor: `filesystem` (`@modelcontextprotocol/server-filesystem`, oficial de Anthropic), el seleccionado en la ficha de investigación.
Cliente: `docs/skills-mcp/tools/mcp-fs.js` - JSON-RPC 2.0 por stdio contra el server oficial

(`initialize` -> `tools/list` -> `tools/call`).
Solicitud realizada:

```
[bloque de codigo]
```

Resultado obtenido: el contenido íntegro del archivo (6.041 caracteres) y, en paralelo, `list_directory` sobre `knowledge-base/` (los 10 archivos de la KB) y `read_text_file` sobre `03_actores_y_roles.md` (3.022 caracteres).
Qué aportó al change: los atributos de `tenant`, `usuario` y `config`; la regla *"Toda tabla de negocio lleva `tenant_id` NOT NULL + FK... Unicidades y búsquedas siempre compuestas"*; el seed; y los nombres reales de los roles. Ese fue el insumo de §5 del spec y de la Decisión 3 del design. También permitió **sustituir una decisión de dominio por una cita de la KB**: el eje 2 salió de la línea *"roles (array enum...)"* de §04, y la alternativa quedó descartada por contradecirla.
Como evidencia de que la herramienta se usó y no sólo se listó: el handshake devuelve `INIT server: secure-filesystem-server 2.0.0` y `READ CHANGES.md -> 33704 chars` a través del protocolo, y `opencode mcp list` muestra **3 servidores conectados** (`context7`, `engram`, `filesystem`).

2.1 Segundo MCP de la sesión: `engram` (memoria persistente)

Solicitud realizada: `mem_save({... project:"odontologia-tpmetodologia", content:"**What**/**Why**/**Where**/**Learned**"})` en cada decisión, bugfix y regla, con `mem_judge` para resolver los candidatos de conflicto que devuelve cada alta.
Resultado obtenido: `{"result":"Memory saved","judgment_required":true}` en cada alta, `{"judgment_status":"judged"}` en cada juicio, y `mem_search` con el filtro de proyecto devuelve las **13 observaciones** del proyecto con sus relaciones y sus pines.
Atar el proyecto: las primeras llamadas devolvieron `unknown_project`, porque el nombre del proyecto no se puede deducir del directorio de trabajo cuando el repositorio está anidado dentro de otro con el mismo remoto Git. La solución son dos piezas: `.engram/config.json` con `{"project_name": "odontologia-tpmetodologia"}` **en el repo** (resuelve por `config`) y `environment: { ENGRAM_PROJECT: "odontologia-tpmetodologia" }` en el server MCP de opencode (resuelve por `process_override`, con la clave `environment` del schema `McpLocalConfig`). Con las dos, una sesión que arranque en este repo escribe y lee memoria del proyecto correcto sin pasar el nombre a mano.
Qué aportó al change: los **gotchas no derivables del código** - que `pydantic-settings` lee `os.environ` incluso en `model_validate`; que `pytest.ini` con `addopts=-q` más un `-q` extra produce `-qq` y oculta el resumen en pytest 9; que `model_validate` no coacciona `str` a `ZoneInfo`; que Postgres admite múltiples valores NULL en un índice único; que el shim `npx.cmd` de Windows parte los argumentos con paths con espacios; y que `openspec validate` no cruza specs entre sí - más un segundo pase de revisión semántica sobre cada decisión. Es aportación al *proceso*, no al *producto*: ninguna línea de código, spec ni ADR salió de ahí.

3. Defectos detectados en los resultados del agente (C-02)

D1 - El spec de C-02 contradecía el contrato de error de `foundation/runt

Gravedad: alta. Contradicción entre dos specs normativos.
Cómo se detectó: cruce del spec nuevo contra el spec ya archivado de C-01, `openspec/specs/foundation/runtime/spec.md:98`:
*"El sistema SHALL responder a todo error con un cuerpo `{code, message, action?}`... **Ningún

endpoint SHALL devolver una forma de error distinta de esta**."*
El spec de C-02 exigía que el error de slug duplicado *"identificara la columna `slug` como causante"* - un dato que no tiene lugar en el envelope. Quien satisfaciera un spec rompería el otro, y ambos validaban.
Fuente o regla: el propio requisito de `foundation/runtime` (R9) y la KB §05, donde se verificó que **ninguno de los 24 RN** (RN-AG-01..07, RN-AU-01..04, RN-CO-01..05, RN-GL-01..04, RN-PA-01..04) cubre el slug duplicado. Por la regla del contrato corresponde el **código genérico documentado**, no inventar un RN.
Antes: `- **THEN** la creación es rechazada por la restricción de unicidad, y el error identifica la columna `slug` como causante`
Después: `- **THEN** la creación es rechazada con un error `409` cuyo cuerpo es `{code, message}` -el código genérico documentado, porque la base de conocimiento no define una regla de negocio para el slug duplicado- y con un mensaje en español-AR que indique que el slug ya está en uso; el detalle técnico de la restricción queda en el log y no en la respuesta`
Se agregó además el escenario 3.3 (*"Todo tenant tiene slug"*), que cierra el hueco que esta corrección destapa.

D2 - La migración `002` dejaba `slug` nullable y rompía el seed del slice a

Gravedad: alta. Defecto de plan de migración, con impacto en datos y en código existente.
Cómo se detectó: dos verificaciones independientes. (a) **Experimento sobre Postgres 16 real**: la premisa implícita del plan era que el índice único garantizaba la unicidad, pero `INSERT INTO exp_slug (nombre, slug) VALUES ('A','a'), ('B',NULL); CREATE UNIQUE INDEX ux_exp_slug ON exp_slug (slug);`
deja insertar la fila con NULL **sin violar el índice**: Postgres admite múltiples valores NULL en un índice único. Con `slug` nullable, un tenant sin slug no dispararía la restricción y "slug único global" quedaría sin garantía. (b) **Cruce con el código existente**: `backend/app/modules/agenda/seed_demo.py:21` inserta el tenant demo **sin `slug`** (`TenantModel(id=TENANT_DEMO, nombre="Consultorio Demo")`), y el plan de C-02 no mencionaba ese archivo en ningún punto: con la columna nullable crea un tenant sin slug, y con la columna NOT NULL rompe el seed con `IntegrityError`.
Fuente o regla: KB §04 (*"Constraints: `slug` kebab-case único global"*), el contrato de unicidad que el propio spec declara, y la regla dura **R6**.
Antes: `ADD COLUMN slug text NULL` -> backfill -> `CREATE UNIQUE INDEX ux_tenant_slug ON tenant (slug)`.
Después: `ADD COLUMN slug text NULL` (nullable **sólo** durante el backfill) -> backfill determinista que **falla con un mensaje explícito** ante colisión -> `ALTER TABLE tenant ALTER COLUMN slug SET NOT NULL` -> `CREATE UNIQUE INDEX ux_tenant_slug ON tenant (slug)`, más la **Decisión 13** nueva en el design y el **upstream obligatorio** de `seed_demo.py` (actualizarlo en este change o diferirlo a C-07 con owner declarado). Se agregaron las tareas 1.10, 1.11 y 1.12.

D3 - El spec filtraba un detalle de implementación

Gravedad: baja.
Cómo se detectó: relectura contra las reglas de redacción de specs de OpenSpec (*"Avoid in specs: internal class/function names, step-by-step implementation details"*).
Antes: `- **THEN** la consulta no devuelve ninguna fila, y la consulta ejecutada incluye la condición de filtro por el `tenant_id` del tenant `A`` - exige verificar el SQL generado.
Después: `- **THEN** no devuelve ninguna fila, y el acceso a datos de negocio no puede

obtenerse sin un tenant, de modo que el filtro por tenant forma parte de la construcción del acceso y no de cada consulta` - la garantía que importa queda en términos observables y la aserción sobre la consulta pasa a las tareas (2.5), donde sí corresponde.

4. Prueba de borde: los límites del validador

La consigna pide, cuando no se detectan defectos, "prueben un caso de borde para evaluar sus límites y documenten el resultado sin inventar un error". Aquí se detectaron defectos, pero la prueba se hizo igual porque evalúa un límite distinto: el del propio mecanismo de validación.
Límite evaluado: `openspec validate` valida cada spec por separado. Con el defecto D1 presente, el validador reportaba el change como válido (`Change 'C-02-tenancy-core-models' is valid`) y `validate --all --strict` también (3/3). El validador comprueba la forma de cada delta -headers, escenarios, `SHALL`- pero **no** cruza el spec nuevo contra los specs archivados, **no** confronta el plan de migración con la semántica real del motor, y **no** revisa el código existente.
Resultado del caso de borde, documentado sin inventar errores:
Comprobación	Resultado
`openspec validate --strict` con D1 presente	"is valid" - **el defecto pasó la validación**
`openspec validate --all --strict`	3/3 - tampoco lo detecta
Cruce manual contra `foundation/runtime`	**contradicción encontrada** (D1)
Experimento de NULLs en índice único sobre Postgres 16	**defecto de plan encontrado** (D2)
Cruce manual con `seed_demo.py`	**omisión encontrada** (D2)
Conclusión honesta: la validación automática es necesaria pero **no suficiente**. Sirve para la forma; no para la coherencia entre specs, ni para el plan de migración frente a la semántica del motor, ni para el cruce con el código. Por eso la verificación cruzada quedó escrita como **tarea 8.6** del change, y no como confianza en la herramienta.

5. Revisión de la ejecución de C-01 (misma consigna, change implement

El agente también produjo resultados para C-01, y la revisión de esa ejecución encontró 5 incidencias. El detalle está en `revision-critica-punto-8.md`; el resumen:
- **D1 (alta)** - el servidor MCP `filesystem` declaraba "CONFIGURADO Y OPERATIVO" sin estar conectado. Causa raíz doble: la entry estaba en `opencode.jsonc` (que el cliente **no lee**) y usaba `npx`, cuyo shim en Windows parte el path del workspace al contener un espacio (`Error: None of the specified directories are accessible`). Corregido con `node` + entry absoluto en el `opencode.json` activo y verificado con `opencode mcp list` y un handshake real.
- **D2 (media)** - violación del spec `foundation/runtime`: el secreto temporal de `dev` se generaba **sin el aviso** que exige el escenario. Corregido y con **prueba de regresión** verificada en rojo sobre el estado previo y en verde sobre el corregido.
- **D3 (baja)** - `design.md` decía "Secrets: ninguno" de forma inexacta para un CI que conecta con Postgres 16 real. Redactado con precisión.
- **D4 (baja)** - contaminación con caracteres CJK en 4 archivos escritos por el agente, detectada con un barrido explícito (`[\u4e00-\u9fff]`) y verificada limpia en el commit.
- **D5 (media)** - el agente introdujo dos CVE moderados al agregar `react-router-dom` 6.x. Actualizado a 7.18.4 y vitest 5.0.5: `npm audit` queda en 0.
Además, la sección 4 de ese informe documenta un **límite del razonamiento** del agente sin inventar errores: en el explore afirmó que el slice archivado era "código validado", lo que era

correcto en su lógica pero se sostenía sobre una suite que corría contra SQLite, donde las claves foráneas no se validan. Al correr contra Postgres 16 aparecieron tres violaciones de FK reales que SQLite nunca detectó.

6. Cómo reproducir la evidencia

```
[bloque de codigo]
```

Archivos corregidos

Archivo	Corrección
`openspec/changes/C-02-tenancy-core-models/specs/tenancy/identidad/spec.md`	D1 (escenario 3.1 reescrito al envelope; nuevo escenario 3.3) y D3 (escenario 1.1 sin detalle de implementacion)
`openspec/changes/C-02-tenancy-core-models/design.md`	D2 (Decision 13, Migration Plan corregido, riesgo nuevo)
`openspec/changes/C-02-tenancy-core-models/tasks.md`	D1 (tarea 4.4), D2 (tareas 1.10, 1.11, 1.12) y verificacion cruzada (tarea 8.6)
`backend/app/core/config.py`, `backend/app/main.py`, `backend/tests/*`	D2 de C-01: aviso del secreto temporal + test de regresion
`frontend/package.json`	D5 de C-01: react-router-dom 7.18.4 y vitest 5.0.5
`openspec/changes/archive/2026-10-19-C-01-foundation-setup-decisiones/design.md`	D3 de C-01: redaccion precisa de "Secrets"
`~/.config/opencode/opencode.json` (fuera del repo)	D1 de C-01: entry del server MCP en el config activo, con `node` + entry absoluto. Backups: `opencode.json.bak`, `opencode.json.bak-c01`
`docs/skills-mcp/mcp-configuracion.md`, `docs/skills-mcp/aplicacion.md`	Estado real del servidor y causa raiz (D1 de C-01)
`docs/skills-mcp/revision-critica-punto-8.md`, `docs/skills-mcp/revision-critica-punto-8-c02.md`	Informes detallados de esta revisión
`docs/skills-mcp/revision-critica.md`, `docs/skills-mcp/TP-revision-critica-puntos-8.pdf`	Este informe y su versión PDF

Conclusión

La evidencia de invocación está documentada con solicitud, resultado y aportación, para la skill y para el MCP, y no como una lista de herramientas instaladas: el artefacto de explore de C-02 es el resultado de la skill, y la lectura de la KB por JSON-RPC es el resultado del MCP. Se detectaron **8 incidencias en total** (3 en los artefactos de C-02 y 5 en la ejecución de C-01), todas corregidas y verificadas: 3 de C-02 con antes/después documentado, 2 de C-01 con prueba de regresión o verificador propio, y el resto con verificación directa en git o con `npm audit`. La prueba de borde documenta, sin inventar errores, que **`openspec validate` no alcanza** para revisar specs: por eso la verificación cruzada quedó como tarea explícita del change.
C-02 queda **preparado para implementar**: 0 de 46 tareas, no implementado, no verificado, no archivado.